

CHATBOT CON RAG

GuaraníBOT

Asistente de inscripción a materias con IA (RAG)

Tecnicatura Superior en Ciencia de Datos e Inteligencia Artificial
Instituto Tecnológico Beltrán

Santiago Mártire

Desarrollo de Sistemas de Inteligencia Artificial · 2026

¿Qué es y qué resuelve?

GuaraníBOT es un chatbot que responde, en lenguaje natural, las dudas que surgen al anotarse a las materias de la carrera.

El problema

- Anotarse suele fallar por cosas concretas: el período aún no abrió, no cumplís las correlativas, o falta un trámite.
- La información está **repartida** entre el plan de estudios, el régimen de correlatividades y el calendario académico.

La solución

- Junta todas esas fuentes y responde al instante, **citando de qué documento salió cada dato**.
- Se apoya en información real del instituto: no improvisa.

¿Qué es RAG?

RAG (*Retrieval-Augmented Generation*) = **buscar primero, redactar después.**

1 · BUSCAR

Busca en los documentos los fragmentos más parecidos a la pregunta.

2 · PASAR CONTEXTO

Le entrega esos fragmentos a un modelo de lenguaje como contexto.

3 · REDACTAR

El modelo redacta la respuesta basándose en ese contexto real.

¿Por qué importa? Como la respuesta se ancla a fragmentos reales y verificables, el bot **evita “alucinar”** (inventar datos) — el gran riesgo de los modelos de lenguaje.

El recorrido de una pregunta



De dónde saca la información

Todo el conocimiento del bot vive en archivos de texto que vos controlás:

documento.md

Base curada: SIU Guaraní, plan de estudios, correlativas y calendario.

Datos/ (calendario)

Calendario académico oficial 2026 del instituto.

Datos/ (correlativas)

Estructura curricular y régimen de correlatividades oficial.

Editás el archivo → volvés a indexar → el bot ya sabe lo nuevo. Sin tocar una línea de código.

ingest.py — preparar la base de búsqueda

Se corre **una vez** (y cada vez que cambian los datos). Convierte los documentos en algo buscable por significado:

- **Corta por encabezados:** cada sección (ej. “Materias de 1° año”) queda en un fragmento propio y completo.
- **Etiqueta la fuente:** a cada fragmento le guarda de qué archivo salió.
- **Calcula los embeddings** (vectores) de cada fragmento.
- **Los guarda en ChromaDB** con métrica coseno, en la carpeta **chroma_db/**.

```
python ingest.py
```

43 fragmentos · 3 archivos

embeddings.py — el “traductor” a vectores

Una computadora no compara significados de palabras: compara **números**. Un modelo de *embeddings* transforma cada texto en un vector.

- Modelo **multilingüe** (entrenado también en español): `paraphrase-multilingual-MiniLM-L12-v2`.
- Cada texto → un vector de **384 números** que representa su significado.
- Corre **100% local** (~220 MB), gratis y sin conexión.
- Clave: **el mismo modelo** se usa para indexar y para preguntar.

Textos con significado parecido → vectores cercanos. Así “materias de primer año” encuentra la sección correcta aunque no use exactamente las mismas palabras.

ChromaDB — encontrar lo relevante

Cuando llega una pregunta, ocurre la parte “Retrieval” de RAG:

- La pregunta se convierte en un vector (mismo modelo del Paso 3).
- ChromaDB compara ese vector con el de todos los fragmentos.
- Usa **similitud del coseno**: qué tan parecido es el significado.
- Devuelve los **8 fragmentos más cercanos** ($K = 8$), con su fuente.

Busca por **significado**, no por palabras exactas: por eso entiende sinónimos y preguntas escritas de mil formas distintas.

Groq + Llama 3.3 — redactar la respuesta

La parte “Generation” de RAG. El modelo de lenguaje recibe el material y escribe la respuesta:

- Se arma un mensaje con los **8 fragmentos** (etiquetados con su fuente) + la pregunta.
- Un **system prompt** le ordena: responder en español, **priorizar el contexto** y **no inventar** si el dato no está.
- El modelo llama-3.3-70b corre en **Groq** (API gratuita y muy rápida).
- La respuesta se muestra **en streaming** (se va escribiendo en vivo).

app.py — la interfaz de chat (Streamlit)

Todo esto se ve como un chat común en el navegador:

- Interfaz de chat simple, con historial de la conversación.
- La respuesta aparece redactada y ordenada, en español.
- Un panel desplegable “**Ver fragmentos usados**” muestra el texto exacto y el archivo de donde salió cada dato.

Ese panel da **trazabilidad**: cualquiera puede verificar de dónde viene la respuesta. Es la mejor prueba de que el bot no inventa.

El desafío técnico que resolví

La primera versión respondía mal en español. La historia del arreglo:

El problema

- El modelo de embeddings por defecto estaba pensado para inglés.
- Ante “materias de primer año”, la sección correcta quedaba en el **puesto 39 de 47**.
- El bot no la veía → respondía “no tengo esa info”.

La solución

- Embeddings **multilingües** (buenos en español).
- Métrica **coseno** + **chunking por encabezados**.
- La misma sección pasó del puesto 39 **al puesto 1**.

10 preguntas para la demo

■ Fechas e inscripción

1. ¿En qué fecha me inscribo a materias del 1er cuatrimestre?
2. ¿Cuándo es el receso invernal en 2026?
3. ¿Hasta cuándo puedo pedir equivalencias de materias?
4. ¿Cuándo son los exámenes finales de febrero?

■ Correlatividades

5. ¿Qué condiciones necesito para anotarme a 2° año?
6. ¿Por qué puede rebotarme el sistema al inscribirme?

■ Plan de estudios

7. ¿Cuáles son las materias de primer año?
8. ¿Qué materias se cursan en tercer año?
9. ¿Qué diferencia hay entre una materia “E” y una “G”?

■ SIU Guaraní

10. ¿Cuáles son los pasos para inscribirme por SIU Guaraní?

En resumen

- Un chatbot que responde con datos reales del instituto...
- ...usando RAG para no inventar...
- ...con embeddings multilingües, ChromaDB y Llama 3.3.

Python

Streamlit

ChromaDB

fastembed

Groq · Llama 3.3

RAG